

Linux / DevOps Project

Linux Production Operations & Self-Healing System

MONITOR → DETECT → RECOVER → VERIFY → REPORT

Cloud Support / Junior DevOps Production Project

1. Problem Statement

Operational Overhead in Production

- Linux servers require continuous checks for resources, services, networking, security, and log errors.
- Manual checks are repetitive and can significantly delay first-level incident response.

2. Project Objective

Automated First-Level Incident Response

- Automate first-level Linux health checks with Bash.
- Detect common operational incidents in real-time.
- Recover only known safe failures automatically.
- Verify and log the recovery results for complete auditability.

3. Real-World Scenario

Enterprise Monitored Infrastructure

- **server1** represents an internal business application server.
- **Monitored components:** Apache HTTPD, BIND DNS, VSFTPD, Chrony NTP, Firewalld, SELinux, File Permissions, and System Logs.

4. Operational Workflow

Automated Execution Pipeline

MONITOR → DETECT → DIAGNOSE → CONTROLLED RECOVERY → VERIFY → LOG & REPORT

5. Project Directory Structure

Standard /opt/ Directory Organization

```
/opt/linux-ops/  
├─ config/          # Config thresholds  
├─ logs/            # linux_ops.log  
├─ reports/         # health_report.txt  
└─ scripts/  
    └─ linux_ops.sh  # Core Production Script
```

6. Monitoring Coverage

System & Network Services Checked

- CPU / RAM / Root Disk Usage
- Apache / BIND / VSFTPD / Chrony Services
- HTTP 80 / DNS 53 / FTP 21 Active Listening Ports
- Gateway Ping & Internet Connectivity Validation
- Recent system errors and critical log entries

7. File & Security Compliance

Hardening & Permission Auditing

- /mydata existence, owner, and group validation
- Permissions and SELinux security context checks
- Firewalld service active state validation
- SELinux verified in Enforcing mode

8. Self-Healing Mechanism

Safe Auto-Recovery Logic

- Detect inactive services via `systemctl`.
- Restart only known safe service failures (e.g., `httpd`).
- Wait briefly and verify the service state.
- Log RECOVERED / RECOVERY FAILED results to logfile.

9. Production Safety Model

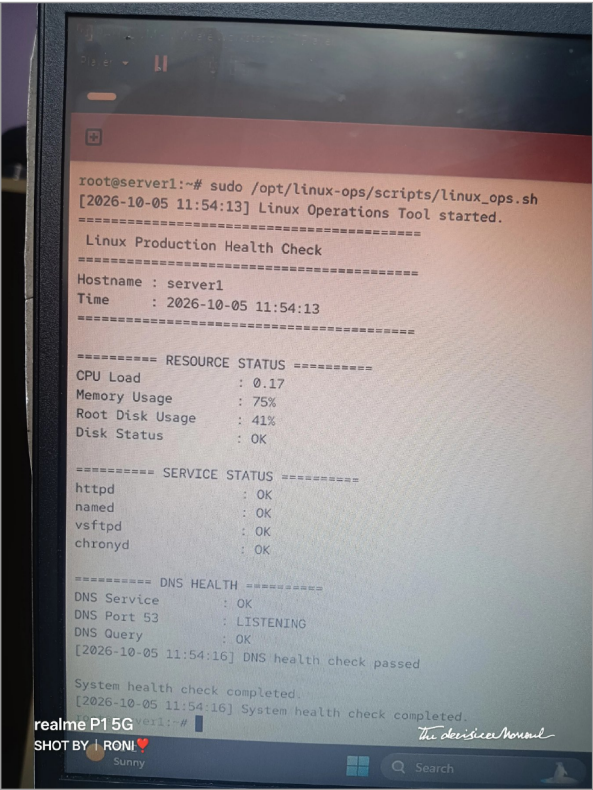
Strict Operational Guardrails

- No blind file deletion.
- No automatic destructive permission changes.
- No disabling SELinux or flushing firewalld rules.
- Risky operations are flagged and reported for administrator review.

Evidence 01 — Health Check & Service Status

Resource and service health checks execution

Screenshot 1: Linux Production Health Check - Script Start & Resource/Service Status
Script run (11:54:13): CPU, memory, disk, httpd/named/vsftpd/chronyd and DNS checks all OK.

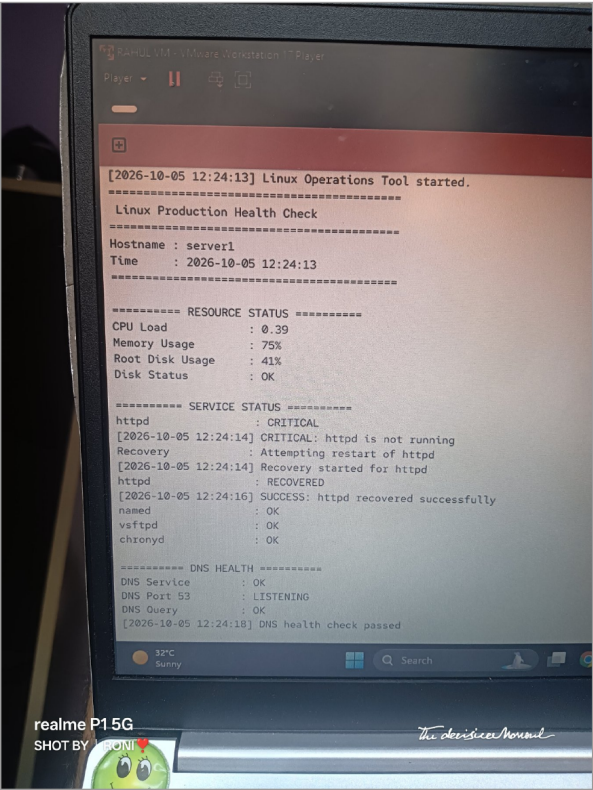


Evidence 02 — Apache Auto-Recovery

httpd failure detected and automatically recovered

Screenshot 2: Auto-Recovery of httpd

httpd detected as CRITICAL, restart attempted automatically and service RECOVERED.

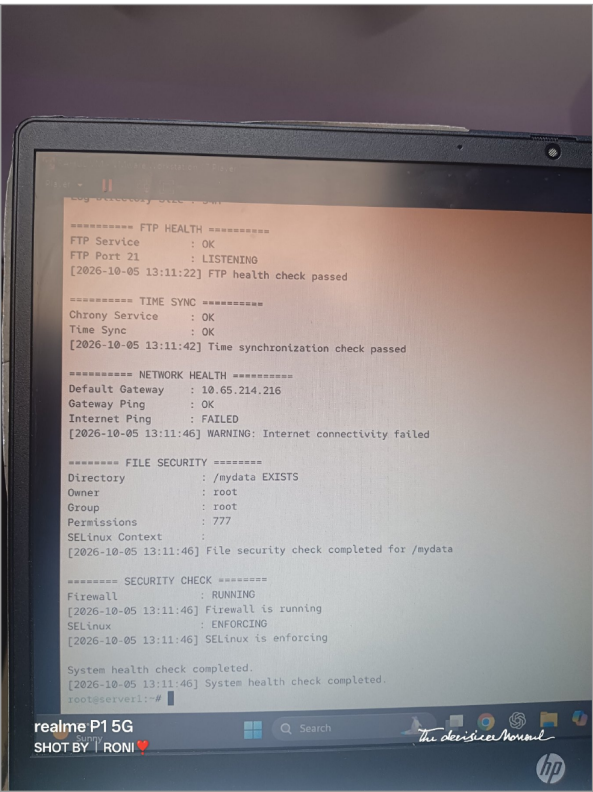


Evidence 03 — Network & Security Checks

Connectivity warning, /mydata security, firewall and SELinux checks

Screenshot 3: Network Warning & Security Checks

Internet ping failure warning, /mydata file security check, firewall running and SELinux enforcing.

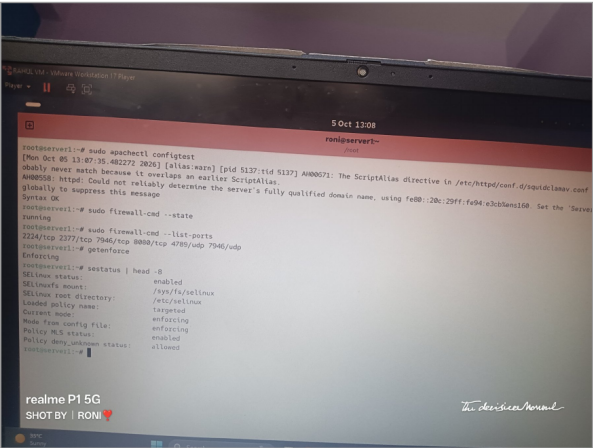


Evidence 04 — Manual Verification Commands

apachectl configtest, firewall-cmd --state, getenforce and sestatus

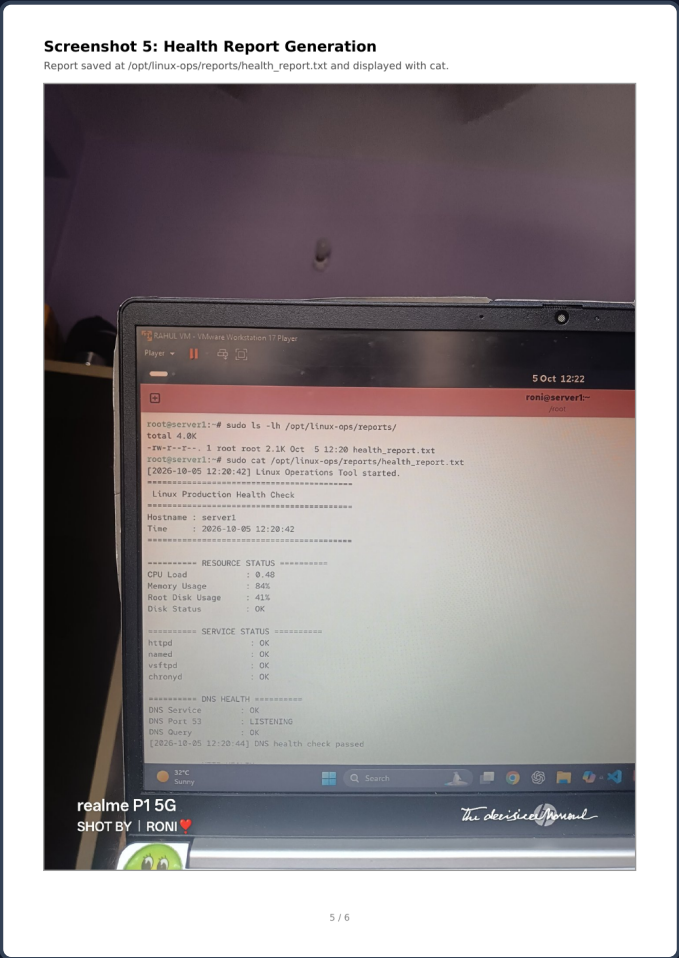
Screenshot 4: Manual Verification Commands

apachectl configtest, firewall-cmd --state, getenforce and sestatus.



Evidence 05 — Health Report Generation

health_report.txt generated and displayed with cat



Evidence 06 — Logs, FTP, Time & Network Health

Log error monitoring, FTP, Chrony time sync and network health

Screenshot 6: Log Monitoring, FTP, Time Sync & Network Health

Recent errors detected from logs; FTP, Chrony time sync and network health checks passed.

